

One Durable Agentic Operator After Capsule Approval

Phase B begins when Phase A has produced a trusted RGY capsule and the user approves it. CubiQo becomes one durable operator: it creates a Duo project, builds a task graph, chooses risk-aware routes, performs safe work, pauses for approvals, logs evidence, writes outcomes, and self-heals through sandboxed proposals.

Approved capsule in

Single durable operator

Vercel Workflow preferred

Supabase truth

trace_id everywhere

Dynamic Duo Mode

User final approval

1

Durable operator

One operator owns the run. Specialist handoffs come later.

2

Route brain

It decides API, MCP, extension, Stagehand/Browserbase, visible browser, computer use, or manual ask.

3

State + trace

Supabase stores dashboard state, audit state, evidence, outcomes, and trace links.

4

Guardrails

No payments. No final send, submit, publish, push, deploy, or irreversible action without the user.

GOVERNING PRINCIPLES

The model proposes. Systems govern.

Phase B is powerful only because the LLM is not allowed to directly invoke tools. Deterministic layers validate, route, approve, budget, log, and stop unsafe work.

1. LLM proposes

- Goal interpretation.
- Task graph suggestions.
- Route recommendations.
- Drafts, questions, and summaries.

2. Deterministic systems govern

- Permissions, connector availability, and routing validity.
- Policy, risk, budget, retry, and security gates.
- Approval gates and irreversible-action blocks.
- RLS, audit logs, trace IDs, and evidence storage.

Canonical control chain: LLM planner -> Decision Router -> Policy Engine -> Execution Engine
-> Evidence Layer.

CAPABILITY ACCESS

Giving CubiQo wings without hardcoding modes

Phase B is not "startup mode" or "Shopify mode." It is access plus routing. When the right playgrounds exist, CubiQo can adapt to new goals through connectors, playbooks, and evidence.

Capability	Unlocks	Boundary
Environment variables	External APIs, provider keys, webhooks.	Validated config; no raw client exposure.
Supabase service role + RLS	Projects, memory, approvals, audit, evidence, dashboard state.	Server-only; user-scoped reads.
Local repo/files	Code inspection, self-heal branches, generated artifacts.	No production push without approval.
Execution environment	Tests, builds, scripts, validation, previews.	Sandbox first; evidence captured.
Stagehand/browser environments	Unknown UI navigation, screenshots, extraction, form prep.	Headless blocked on risky platforms.
OAuth connector registry	User-approved platform access.	Scopes tracked; missing access becomes a Duo request.
Model endpoints	Reasoning, vision, search, embeddings, computer-use fallback.	Budget governor and output validation.

What hands off, what Phase B produces

Phase A hands off

- RGY capsule: signal_id, keyword, domain, color.
- memory_events and user_ai_profile.
- signals activity stream.
- Safety classification cleared, not crisis/red blocked.

Phase B produces

- duo_projects, duo_tasks, duo_task_edges.
- duo_approvals, artifacts, external refs, timeline events.
- duo_tool_calls, duo_outcomes, stream_events.
- self_reports and self_heal_proposals.

CANONICAL PHASE B PIPELINE

From capsule to completed goal

Phase B is execution-aware, but not reckless. Every step is planned, routed, logged, trace-linked, and gated by risk. CubiQo can act broadly because the operating system around the model constrains how it acts.

Approved Capsule

User shoots/approves an RGY capsule or states an explicit goal.

Durable Operator

Vercel Workflow preferred for pause/resume, retries, and long-running steps.

Duo Project

Supabase project shell, trace_id, status, owner, success criteria, and timeline.

Goal Interpreter

Defines outcome, constraints, success criteria, risks, and missing info.

Task Graph

Breaks work into steps, dependencies, blockers, questions, and approval gates.

Decisioning

Chooses API, MCP, extension, Stagehand/Browserbase, computer use, code, or manual route.

Playgrounds

Supabase, Workflow, code, browser, Vercel, APIs, design/media, analytics, AI providers.

Duo Mode

Dynamic project dashboard with tasks, blockers, approvals, artifacts, and logs.

Approval Gates

User presses the final button for external publish/send/submit/deploy/payment-related actions.

Learn + Heal

Report progress, capture outcomes, retry failures, propose patches, improve future routes.

PHASE B STEP BREAKDOWN

This is the engineering/BA split. Each step has its own state, acceptance tests, trace records, and UI surface.

Bifurcated
implementation
steps

Step	Name	Requirement	Truth / Runtime
B0	Entry gate	Approved capsule or explicit goal creates trace_id.	Supabase + app route
B1	Durable operator	Single operator run with pause, resume, retry, timeout handling.	Vercel Workflow preferred
B2	Duo project	Create project shell, owner, status, success criteria, first timeline event.	Supabase
B3	Goal interpretation	Clarify outcome, missing facts, constraints, risks, access, done definition.	Operator + LLM
B4	Task graph	Build tasks, dependencies, blockers, questions, artifacts, approvals.	Supabase tasks
B5	Decision router	Score API/MCP/extension/Stagehand/browser/computer/manual routes.	Supabase tool call trace
B6	Connector readiness	Check credentials, scopes, setup needs, status, and source of truth.	Connector registry
B7	Tool execution	Run safe read/draft/sync/write steps with idempotency and evidence.	Workflow step + Supabase
B8	Approval gate	Pause before final external actions or sensitive writes.	Reuse action_approvals if working
B9	Evidence timeline	Store screenshots, logs, API responses, external IDs, artifacts.	Supabase + storage
B10	Outcome writeback	Mark work done/blocked/failed, write outcomes, update memory.	Supabase outcomes

Step	Name	Requirement	Truth / Runtime
B11	Self-report	Daily report on crons, cost, failures, active projects, blocked tasks.	Cron/workflow
B12	Self-heal proposal	Diagnose repeated failures in sandbox/branch, propose patch and tests.	Sandbox + approval
B13	Specialist handoffs later	Add specialist workers only after the single operator contract is stable.	Later

DELIVERY SLICES

The work should not ship as one huge block. Each slice has a visible done condition and can be tested independently.

How Phase B should be built

Slice	Steps	Done When
B-Core	B0-B5	Capsule -> Duo Mode -> project created -> task graph visible -> route selected.
B-Execution	B6-B10	Task survives restart, executes, logs evidence, renders approval card, writes outcome.
B-Ops	B11-B12	Daily self-report exists, failing work is detected, self-heal proposal appears with diff/tests.
B-Scale	B13+	Specialist handoffs and public beta commercial packaging after core execution is stable.

VISUAL FLOW

**Power
agentic
operating
model**

The left lane is the user-approved capsule. The main lane is one durable CubiQo operator: start workflow, create trace-linked state, plan, route, execute safely, communicate in Duo Mode, and learn from the outcome.



**CONNECTOR +
PLAYGROUND
REGISTRY**

**SUPABASE
DUO PROJECT**
trace_id

**QUESTIONS
BLOCKERS
ACCESS NEEDS**

**SAFE EXECUTION
DRAFT / READ
SYNC / PREP**

**APPROVAL
REQUESTS
FINAL GATES**

**ARTIFACTS
LOGS
EVIDENCE**

ROUTES THE SINGLE OPERATOR CAN CHOOSE IN REAL TIME

**OFFICIAL
API**

**PLUGIN
MCP**

**BROWSER
EXTENSION**

**STAGEHAND
BROWSERBASE**

**VISIBLE /
HEADLESS
LOW RISK**

**COMPUTER
USE**

**MANUAL
ASK**

**SELF REPORT
CRON + COST
HEALTH DAILY**

**SELF HEAL
BRANCH PATCH
TEST + PROPOSE**

**CONVERSION
SHARE / REFERRAL
RESULT CARDS**

**CODEBASE EYES
REPO / TESTS
DEPLOYS**

**LEARN
OUTCOMES
PLAYBOOKS**

**PHASE B PRINCIPLE: ONE DURABLE OPERATOR CAN WORK BROADLY ONLY
BECAUSE EVERY ACTION HAS ROUTE SCORING, `trace_id`, LOGGING, EVIDENCE,
AND APPROVAL GATES.**

DECISIONING LAYER

How CubiQo chooses the optimal method

The decisioning layer prevents CubiQo from going in circles or picking risky automation. It scores every possible route before acting.

Route preference order

1. Official API

Best when user OAuth/token exists and platform supports the action.

2. Plugin / MCP

Structured methods with typed inputs and observable outputs.

3. Extension

Best for authenticated user-session workflows where headless is risky.

4. Stagehand / Browserbase

Preferred unknown-UI browser layer before raw headless scripts.

5. Visible browser

User can watch, intervene, and press final buttons.

6. Headless

Only for low-risk public/owned/admin pages.

7. Computer use

For unknown visual UI when DOM paths are unavailable.

8. Manual ask

When risk, missing auth, or policy boundaries make automation unsafe.

Scoring factors

- Reliability and success evidence.
- Auth availability and connector status.
- Platform policy and account blocking risk.
- Data sensitivity and privacy risk.
- Reversibility and final-action impact.
- User approval need.
- Provider/API/browser cost.
- Speed and timeout risk.
- Whether the method can prove it worked.
- Loop risk and escalation threshold.

CONNECTOR READINESS

CubiQo can be task-agnostic only if its connector registry is structured. New tools can be onboarded, but the operating categories and access request mechanism must already exist.

What needs to be ready in advance

Category	Examples	Phase B purpose
Database	Supabase	Audit/state/dashboard truth: projects, tasks, approvals, logs, memory, sync refs.
Durable runtime	Vercel Workflow	Preferred pause/resume/retry runtime for long-running operator steps.
Code/Git	GitHub, local repo, test runner	Codebase answers, self-healing, PRs, build diagnostics.
Deploy	Vercel	Bulk domains, preview deployments, logs, domain/SSL status.
Browser	Stagehand/Browserbase, visible browser, low-risk headless, computer use	Unknown UI, screenshots, form prep, site inspection.
Extension	Authenticated browser bridge	Safer session-aware automation where headless is risky.
Commerce	Shopify, Printify, Printful, Gelato	POD setup, product drafts, listing status, fulfillment refs.
Social	LinkedIn, X, Instagram, TikTok, YouTube, Reddit, Bluesky	Drafts, approvals, schedules, analytics, platform-specific status.
Design/media	GFX tools, Canva, image/video tools	Marketing assets, mockups, media generation and review.
AI providers	OpenAI, Claude, Gemini, Grok, Llama providers	Response comparison, cost/latency tracking, saved answers.
Email/forms	Resend, SendGrid, Postmark	Contact forms, notifications, draft outreach, transactional mail.
Analytics	Vercel Analytics, PostHog, GA/Plausible	Domain performance, conversion, usage, product loops.

Category	Examples	Phase B purpose
Events/local	Search, calendars, Meetup/Eventbrite-like sources	Local events, social options, reminders, RSVP notes.

GOVERNORS

What stops CubiQo from becoming chaotic

These are deterministic gates around the operator. They are not optional polish; they are what make broad agentic execution safe enough to ship.

Budget governor

- Checks estimated cost before paid external calls.
- Writes `budget_events` with `trace_id` and provider.
- Pauses when project/user/provider limits are hit.
- Uses fair-use / agentic credits, not vague unlimited.

Security breaker

- Blocks scope exceeded, rate spikes, cost spikes.
- Flags prompt injection and exfiltration risk.
- Kills repeated failure loops.
- Blocks headless on risky platforms.

Policy engine

- Validates tool call schemas before execution.
- Checks connector scopes and action allowlists.
- Requires approval for irreversible steps.
- Writes audit records with route and evidence.

The dashboard should feel alive while preventing two tasks from editing the same outside resource.

How Duo Mode stays live and safe

Streaming + realtime

- `stream_events` are append-only and trace-linked.
- Frontend subscribes by `project_id`.
- Timeline updates as tasks plan, execute, pause, fail, recover.
- Approval cards appear from data, not hardcoded screens.

Resource locks + concurrency

- Acquire a lock before any external write.
- No two tasks write the same Shopify product, Git branch, or deployment at once.
- Max active projects per user and max parallel tasks per project.
- Anti-loop: max attempts per route and total attempts per task.

Dynamic dashboard, not hardcoded pages

Project core

- Goal summary and success criteria.
- trace_id and workflow run id.
- Task graph with dependencies.
- Questions CubiQo needs answered.
- Blockers and risks.
- Status and timeline.

Work surface

- Access requests.
- Approval requests.
- Existing action_approvals/action_audit_logs if working.
- Artifacts and drafts.
- External IDs and sync logs.
- Notes and reminders.

Context variants

- Shopify/POD product launch.
- Marketing content calendar.
- Bulk domain deployment.
- AI response comparison.
- Startup/investor workflow.

What gives CubiQo power

Eyes

- Vision input for screenshots, image uploads, camera frames.
- Browser state: URL, screenshot, DOM/accessibility tree, page text.
- Stagehand/Browserbase eyes for unknown UI and browser evidence.
- Computer-use observation for unknown UI.
- Codebase eyes: repo index, git diff, tests, logs, deploys.
- Runtime eyes: cron, queues, API cost, errors, retries, health.

Playgrounds

- Supabase DB playground.
- Vercel Workflow durable runtime playground.
- Code/Git sandbox.
- Stagehand/Browserbase, browser, extension, low-risk headless playgrounds.
- Vercel deployment playground.
- API dry-run playground.
- Design/media and AI comparison playgrounds.

EXAMPLE SCENARIOS

Phase B must support these without becoming hardcoded

Scenario	Agentic capability	Primary data source
Shopify POD	Products, mockups, apps, pricing, listing readiness, launch checklist.	Supabase coordination; Shopify/POD as source of truth.
Marketing posts	Ideas, assets, drafts, review queue, schedule, captions, platform status.	Supabase workflow; social platforms and GFX tools external.
Bulk domains	Domain list, templates, DNS, Vercel deploys, SSL, forms, leads, analytics.	Supabase config; Vercel/DNS/email/analytics external.
AI comparison	One prompt, multiple provider widgets, cost, latency, saved responses.	Supabase logs; providers external APIs.
Social/community	Find events, comfort filters, reminders, notes, RSVP status.	Supabase saved options; event/search sources external.
Chatroom/matching	Consent, preferences, moderation, safety flags, admin review.	Supabase app data; extra care for sensitive media/identity.
Startup/investment	Investor CRM, pitch assets, grants, deadlines, outreach drafts, follow-ups.	Supabase project state; web/search/provider sources external.

CubiQo reports and repairs itself

Codebase awareness

CubiQo can answer where features live, what changed, why tests fail, which routes touch memory, and what remains hardcoded.

Daily self-report

Cron health, background failures, capsules created, API cost, slow routes, deploy status, and open self-heal suggestions.

Self-healing

Detect, diagnose, patch in branch/sandbox, run tests, present evidence, and ask before merge or deploy.

CHECKLIST

Phase B requirement checklist

Workstream	Requirement	Status
Agentic engine	Approved capsule launches one durable operator run, not a multi-agent swarm.	Core
Durable runtime	Vercel Workflow is the preferred runtime for pause/resume/retryable operator steps.	Core
Supabase truth	Projects, tasks, tool calls, approvals, audit logs, timeline, outcomes, and dashboard state live in Supabase.	Core
Trace contract	trace_id links projects, tasks, tool calls, approvals, timeline events, outcomes, and self-heal proposals.	Core
Decisioning	Route planner scores API, MCP/plugin, extension, Stagehand/Browserbase, visible browser, headless, computer use, and manual ask.	Core
Connector registry	All tools expose auth type, actions, risk, status, setup needs, and source of truth.	Core
New tools	Tool onboarding protocol supports user-requested unknown tools in real time.	Core
Budget governor	Paid external calls check project/user/provider limits before execution and write budget_events.	Core
Security breaker	Scope, rate, prompt-injection, cost spike, exfiltration, and repeated-failure breakers can pause/block/kill work.	Core
Realtime stream	stream_events feed Duo Mode timeline, task updates, approval cards, blockers, and evidence state.	Core
Resource locks	External writes require locks so parallel tasks cannot mutate the same external object.	Core
Concurrency	Max active projects, max parallel tasks, connector rate limits, and anti-loop limits are enforced.	Core
Playgrounds	DB, Workflow, code, Stagehand/Browserbase, extension, deploy, API, design, AI, and analytics playgrounds exist.	Core

Workstream	Requirement	Status
Duo Mode	Dashboard is schema-driven and changes by goal/domain/provider/status.	Core
Approvals	Reuse existing action_approvals and action_audit_logs if working; final external actions require explicit approval.	Guardrail
Codebase awareness	CubiQo can answer real-time questions about its own repo, tests, deploys, and docs.	Core
Self-reporting	Daily reports cover cron health, cost, failures, capsules, recommendations, and deploy state.	Core
Self-healing	Failures create retry, diagnosis, patch proposal, tests, and approval-gated deployment.	Core
Conversion	Shareable results, referral loops, saved progress, and "what CubiQo did" summaries.	Expansion
Startup success	Investor CRM, grants, pitch assets, landing pages, analytics, outreach drafts.	Expansion
Public beta monetisation	Fair-use / agentic credits first; Stripe is a public-beta gate, not an engineering entry gate.	Later
Specialist handoffs	Add specialist workers only after the single durable operator contract is stable.	Later
Tier 4 affiliates	Impact/CJ/ShareASale scale-up.	Later
Amazon PA API	Amazon product cards and ASIN lookup.	Later

ACCEPTANCE CRITERIA

What each slice must prove

B-Core

Capsule opens Duo Mode, project is created, trace_id is assigned, task graph is visible, route is selected with rejection reasons.

B-Execution

Queued task survives restart, checks connector readiness, executes safe steps, logs evidence, pauses for approval, and writes outcome.

B-Ops

Daily self-report is generated, failed cron/background work is detected, self-heal proposal is created in sandbox/branch with tests.

B-Scale

Specialist handoffs, broader connector networks, and Stripe/public beta packaging wait until core and execution are stable.

NORTH STAR

What Phase B should feel like

CubiQo reads everything it already knows before asking a question, builds the task graph, checks access, scores routes, chooses the safest reliable path, executes reversible work, pauses before final actions, shows exactly what will happen, preserves artifacts even on cancellation, tracks cost, learns from outcomes, reports its own health, and proposes fixes in a branch with user sign-off. It never auto-publishes, never auto-sends, and never touches production without the user pressing the button.

SINGLE SOURCE

How to use this doc

This file is the Phase B visual companion to the Phase A master. Use it to discuss power agentic requirements, implementation tickets, Duo Mode behavior, connector readiness, and testing scope.